x

Cross-Tool Practice Set — Solutions

Every numeric answer here was computed by executing the code against `sales_practice.csv` in all three tools. Excel, DuckDB SQL and PySpark agree to the cent.

Setup — define these first.

```sql
-- SQL (DuckDB / adapt schema names for SQL Server)
CREATE VIEW raw AS SELECT * FROM read_csv_auto('sales_practice.csv', header=true);
CREATE VIEW sales AS
SELECT *, INITCAP(TRIM(Region)) AS RegionClean,
       Units*UnitPrice*(1-DiscountPct) AS NetRevenue,
       Units*UnitCost AS TotalCost
FROM (SELECT DISTINCT * FROM raw);
```

```python
# PySpark
from pyspark.sql import SparkSession, functions as F, Window
spark = SparkSession.builder.appName("practice").master("local[*]") \
        .config("spark.sql.shuffle.partitions","8").getOrCreate()
raw = spark.read.csv("sales_practice.csv", header=True, inferSchema=True)
s = (raw.dropDuplicates()
       .withColumn("RegionClean", F.initcap(F.trim(F.col("Region"))))
       .withColumn("NetRevenue", F.col("Units")*F.col("UnitPrice")*(1-F.col("DiscountPct")))
       .withColumn("TotalCost", F.col("Units")*F.col("UnitCost")))
```

In Excel, `Data` is the raw sheet (rows 2:6041) and `DataClean` is deduplicated (rows 2:6001). Column letters: **A** OrderID, **B** OrderDate, **C** Region, **E** SalesRep, **G** Category, **J** Channel, **K** CustomerSegment, **L** CustomerID, **M** Units, **N** UnitPrice, **O** DiscountPct, **P** UnitCost, **Q** ShipDays, **R** Returned.

Section A — Data Quality

A1 — Raw row count · 6,040

```
Excel  =COUNTA(Data!A2:A6041)
SQL    SELECT COUNT(*) FROM raw;
Spark  raw.count()
```

A2 — After deduplication · 6,000 (40 duplicate rows)

```
Excel  Data > Remove Duplicates (all columns)  |  =COUNTA(DataClean!A2:A6001)
SQL    SELECT COUNT(*) FROM (SELECT DISTINCT * FROM raw);
Spark  raw.dropDuplicates().count()
```

A3 — Negative Units · 30

```
Excel  =COUNTIF(Data!M2:M6041,"<0")
SQL    SELECT COUNT(*) FROM raw WHERE Units < 0;
Spark  raw.filter(F.col("Units") < 0).count()
```

A4 — Blank ShipDays · 91

```
Excel  =COUNTBLANK(Data!Q2:Q6041)
SQL    SELECT COUNT(*) FROM raw WHERE ShipDays IS NULL;
Spark  raw.filter(F.col("ShipDays").isNull()).count()
```

A5 — Blank CustomerSegment · 60

```
Excel  =COUNTBLANK(Data!K2:K6041)
SQL    SELECT COUNT(*) FROM raw WHERE CustomerSegment IS NULL;
Spark  raw.filter(F.col("CustomerSegment").isNull()).count()
```

A6 — Region variants · 12 in SQL/Spark

`East`, `North`, `South`, `West`, `EAST`, `NORTH`, `SOUTH`, `WEST`, and the same four again with a **leading space**.

Two separate defects: inconsistent casing, and leading whitespace. They need different fixes (`UPPER` / `INITCAP` vs `TRIM`), which is

why the cleaning expression needs both.

```
SQL    SELECT DISTINCT Region FROM raw ORDER BY 1;
Spark  raw.select("Region").distinct().orderBy("Region").show(20, False)
Excel  Filter the Region column dropdown and read the list
```

A7 — Rows in a duplicated OrderID · 80 (40 pairs)

```
Excel  =SUMPRODUCT((COUNTIF(Data!A2:A6041,Data!A2:A6041)>1)*1)
SQL    SELECT COUNT(*) FROM raw WHERE OrderID IN
         (SELECT OrderID FROM raw GROUP BY OrderID HAVING COUNT(*) > 1);
Spark  d = raw.groupBy("OrderID").count().filter("count > 1").select("OrderID")
       raw.join(d, "OrderID").count()
```

A8 — Cleaned Region expression

```
Excel  =PROPER(TRIM(C2))
SQL    INITCAP(TRIM(Region))          -- SQL Server: UPPER(LEFT(...,1)) + LOWER(SUBSTRING(...))
Spark  F.initcap(F.trim(F.col("Region")))
```

A9 — Handling strategy

Not the same strategy for all three, and saying so is the point.

- **Negative Units (30)** — impossible values, almost certainly data-entry or reversal artefacts. Investigate first; if they represent returns they should be reclassified, not deleted. Never silently `ABS()` them.
- **Blank ShipDays (91)** — missing measurement, not missing fact. Impute with the **median** (robust to the right tail that shipping times have), or keep as `Unknown` if downstream logic can handle it. Do not fill with 0 — that asserts same-day delivery.
- **Blank CustomerSegment (60)** — missing categorical. Fill with an explicit `"Unknown"` category rather than dropping the rows, so order counts stay correct. Dropping would bias every segment-level metric.

Section B — Basic Aggregation

B1 — Total net revenue · 340,400,414.67

```
Excel  =SUMPRODUCT(DataClean!M2:M6001,DataClean!N2:N6001,1-DataClean!O2:O6001)
SQL    SELECT ROUND(SUM(NetRevenue),2) FROM sales;
Spark  s.agg(F.round(F.sum("NetRevenue"),2)).show()
```

B2 — Revenue by region

Region	Net Revenue
South	89,480,462.92
West	85,762,222.61
East	83,369,662.52
North	81,788,066.63

```
SQL    SELECT RegionClean, ROUND(SUM(NetRevenue),2) r FROM sales GROUP BY 1 ORDER BY r DESC;
Spark  s.groupBy("RegionClean").agg(F.round(F.sum("NetRevenue"),2).alias("r")).orderBy(F.desc("r")).show()
Excel  Pivot: RegionClean in Rows, Sum of NetRevenue in Values, sort descending
```

B3 — Revenue and order count by category

Category	Net Revenue	Orders
Electronics	174,848,038.03	1,596
Furniture	91,648,978.36	1,674
Appliances	71,827,858.11	1,136
Office Supplies	2,075,540.18	1,594

Note Office Supplies has nearly as many orders as Electronics but 1/84th the revenue — order count and revenue tell completely different stories.

```
SQL    SELECT Category, ROUND(SUM(NetRevenue),2) r, COUNT(*) n FROM sales GROUP BY 1 ORDER BY r DESC;
Spark  s.groupBy("Category").agg(F.round(F.sum("NetRevenue"),2).alias("r"), F.count("*").alias("n")).orderBy(F.desc("r")).show()
```

B4 — Total units · 20,919

```
Excel  =SUM(DataClean!M2:M6001)
SQL    SELECT SUM(Units) FROM sales;
Spark  s.agg(F.sum("Units")).show()
```

B5 — South revenue · 89,480,462.92

```
Excel  =SUMPRODUCT((TRIM(UPPER(DataClean!C2:C6001))="SOUTH")*DataClean!M2:M6001
              *DataClean!N2:N6001*(1-DataClean!O2:O6001))
SQL    SELECT ROUND(SUM(NetRevenue),2) FROM sales WHERE RegionClean = 'South';
Spark  s.filter(F.col("RegionClean")=="South").agg(F.round(F.sum("NetRevenue"),2)).show()
```

> A plain `SUMIFS(...,"South")` in Excel **would** catch `SOUTH` (COUNTIF/SUMIFS are case-insensitive) but **would not** catch `" South"` with the leading space. The `TRIM` is what matters here, not the `UPPER`.

B6 — Online + Corporate + 2025 · 16,871,793.73

```
Excel  =SUMPRODUCT((DataClean!J2:J6001="Online")*(DataClean!K2:K6001="Corporate")
              *(YEAR(DataClean!B2:B6001)=2025)*DataClean!M2:M6001
              *DataClean!N2:N6001*(1-DataClean!O2:O6001))
SQL    SELECT ROUND(SUM(NetRevenue),2) FROM sales
       WHERE Channel='Online' AND CustomerSegment='Corporate' AND YEAR(OrderDate)=2025;
Spark  s.filter((F.col("Channel")=="Online") & (F.col("CustomerSegment")=="Corporate")
                & (F.year("OrderDate")==2025)).agg(F.round(F.sum("NetRevenue"),2)).show()
```

B7 — Avg units, Electronics, not returned · 3.4949

```
Excel  =AVERAGEIFS(DataClean!M2:M6001,DataClean!G2:G6001,"Electronics",DataClean!R2:R6001,"No")
SQL    SELECT ROUND(AVG(Units),4) FROM sales WHERE Category='Electronics' AND Returned='No';
Spark  s.filter((F.col("Category")=="Electronics")&(F.col("Returned")=="No")).agg(F.avg("Units")).show()
```

B8 — Distinct reps in South · 3

```
Excel  =SUMPRODUCT((COUNTIFS(DataClean!C2:C6001,"South",DataClean!E2:E6001,DataClean!E2:E6001)>0)
              /COUNTIF(DataClean!E2:E6001,DataClean!E2:E6001))
SQL    SELECT COUNT(DISTINCT SalesRep) FROM sales WHERE RegionClean='South';
Spark  s.filter(F.col("RegionClean")=="South").select("SalesRep").distinct().count()
```

B9 — Max single order per region

East 558,600.14 · North 483,302.10 · South 673,191.45 · West 580,355.08

```
SQL    SELECT RegionClean, ROUND(MAX(NetRevenue),2) FROM sales GROUP BY 1 ORDER BY 1;
Spark  s.groupBy("RegionClean").agg(F.round(F.max("NetRevenue"),2)).orderBy("RegionClean").show()
Excel  =SUMPRODUCT(MAX((TRIM(UPPER(DataClean!C2:C6001))="SOUTH")*DataClean!M2:M6001
              *DataClean!N2:N6001*(1-DataClean!O2:O6001)))
```

> Wrapping `MAX` in `SUMPRODUCT` forces array evaluation without Ctrl+Shift+Enter. Worth memorising.

B10 — Orders with any discount · 2,435 (of 6,000)

```
Excel  =COUNTIF(DataClean!O2:O6001,">0")
SQL    SELECT COUNT(*) FROM sales WHERE DiscountPct > 0;
Spark  s.filter(F.col("DiscountPct") > 0).count()
```

B11 — Distinct customers · 897

```
Excel  =SUMPRODUCT(1/COUNTIF(DataClean!L2:L6001,DataClean!L2:L6001))
SQL    SELECT COUNT(DISTINCT CustomerID) FROM sales;
Spark  s.select("CustomerID").distinct().count()
```

Section C — Business Metrics

C1 — Return rate by category

Appliances 6.78% · Electronics 7.96% · Furniture 7.53% · Office Supplies 6.02%

```
SQL    SELECT Category, ROUND(AVG(CASE WHEN Returned='Yes' THEN 1.0 ELSE 0 END),4)
       FROM sales GROUP BY 1 ORDER BY 1;
Spark  s.groupBy("Category").agg(F.round(F.avg((F.col("Returned")=="Yes").cast("double")),4)).orderBy("Category").show()
Excel  =COUNTIFS(DataClean!G2:G6001,"Electronics",DataClean!R2:R6001,"Yes")
              /COUNTIF(DataClean!G2:G6001,"Electronics")
```

C2 — Gross margin % by category

Appliances 39.07% (highest) · Electronics 37.79% · Furniture 37.58% · Office Supplies 37.04%

```
SQL    SELECT Category, ROUND(SUM(NetRevenue-TotalCost)/SUM(NetRevenue),4)
       FROM sales GROUP BY 1 ORDER BY 2 DESC;
Spark  s.groupBy("Category").agg(F.round((F.sum("NetRevenue")-
F.sum("TotalCost"))/F.sum("NetRevenue"),4)).orderBy(F.desc("...")).show()
```

Note this is `SUM(margin)/SUM(revenue)` — a **revenue-weighted** margin, not the average of per-order margins. Those are different numbers and you should say which you computed.

C3 — Average ShipDays by channel

Online 6.5306 · Partner 6.4623 · Retail 6.6181

```
Excel  =AVERAGEIFS(DataClean!Q2:Q6001,DataClean!J2:J6001,"Online")
SQL    SELECT Channel, ROUND(AVG(ShipDays),4) FROM sales GROUP BY 1 ORDER BY 1;
Spark  s.groupBy("Channel").agg(F.round(F.avg("ShipDays"),4)).orderBy("Channel").show()
```

All three ignore blanks, dividing by the count of non-null values (1,977 of 2,003 for Online). That's the correct default here — but it's a decision, not an accident, and you should be able to state it.

C4 — Top 5 reps by revenue

Priya Iyer 31,678,014.81 · Karan Joshi 31,422,677.98 · Rahul Banerjee 30,695,509.36 · Divya Menon 29,333,764.59 · Amit Verma 28,489,564.40

```
SQL    SELECT SalesRep, ROUND(SUM(NetRevenue),2) r FROM sales GROUP BY 1 ORDER BY r DESC LIMIT 5;
Spark  s.groupBy("SalesRep").agg(F.round(F.sum("NetRevenue"),2).alias("r")).orderBy(F.desc("r")).show(5)
```

C5 — Revenue by region and year

Region	2024	2025
East	39,902,935.21	43,466,727.32
North	41,668,189.15	40,119,877.48
South	42,769,976.18	46,710,486.74
West	45,885,570.85	39,876,651.75

```
SQL    SELECT RegionClean, YEAR(OrderDate) y, ROUND(SUM(NetRevenue),2) FROM sales GROUP BY 1,2 ORDER BY 1,2;
Spark  s.groupBy("RegionClean").pivot("Year", [2024,2025]).agg(F.round(F.sum("NetRevenue"),2)).show()
       # after: s = s.withColumn("Year", F.year("OrderDate"))
Excel  Pivot: RegionClean in Rows, Year in Columns, Sum of NetRevenue in Values
```

C6 — Top three months

2024-11 22,237,653.00 · **2025-11** 21,148,994.02 · **2024-10** 20,391,616.15

All three are Oct–Nov, in both years — a repeating **seasonal peak** in Q4, not a one-off spike.

```
SQL    SELECT strftime(OrderDate,'%Y-%m') m, ROUND(SUM(NetRevenue),2) r
       FROM sales GROUP BY 1 ORDER BY r DESC LIMIT 3;
Spark  s.groupBy(F.date_format("OrderDate","yyyy-
MM").alias("m")).agg(F.round(F.sum("NetRevenue"),2).alias("r")).orderBy(F.desc("r")).show(3)
```

C7 — Discounted vs not

Group	Orders	Avg Units	Revenue
No discount	3,565	3.482	209,505,433.45
Discounted	2,435	3.493	130,894,981.22

Average units are essentially identical (3.482 vs 3.493). Discounting did **not** meaningfully increase order size here — so the discount is pure margin give-away in this dataset. That's the analytical point of the question.

```
SQL    SELECT CASE WHEN DiscountPct=0 THEN 'None' ELSE 'Discounted' END d,
              COUNT(*), ROUND(AVG(Units),3), ROUND(SUM(NetRevenue),2) FROM sales GROUP BY 1;
Spark  s.withColumn("d", F.when(F.col("DiscountPct")==0,"None").otherwise("Discounted")) \
        .groupBy("d").agg(F.count("*"), F.round(F.avg("Units"),3), F.round(F.sum("NetRevenue"),2)).show()
```

C8 — ShipDays buckets

1-3: **1,449** · 4-6: **1,474** · 7-9: **1,503** · 10+: **1,484** · Unknown: **90**

```
SQL    SELECT CASE WHEN ShipDays IS NULL THEN 'Unknown' WHEN ShipDays<=3 THEN '1-3'
              WHEN ShipDays<=6 THEN '4-6' WHEN ShipDays<=9 THEN '7-9' ELSE '10+' END b,
              COUNT(*) FROM sales GROUP BY 1 ORDER BY 1;
Spark  s.withColumn("b", F.when(F.col("ShipDays").isNull(),"Unknown")
                        .when(F.col("ShipDays")<=3,"1-3").when(F.col("ShipDays")<=6,"4-6")
                        .when(F.col("ShipDays")<=9,"7-9").otherwise("10+")) \
        .groupBy("b").count().orderBy("b").show()
Excel  Pivot: ShipDays in Rows > right-click > Group > Start 1, End 12, By 3
```

> **The null check must come first** in the `when` chain. If `ShipDays<=3` is evaluated first, nulls fall through to `otherwise` and get silently counted as `10+`.

C9 — 2025 minus 2024 · −52,928.10

Essentially flat year on year — a rounding-error difference on 340M.

```
SQL    SELECT ROUND(SUM(CASE WHEN YEAR(OrderDate)=2025 THEN NetRevenue ELSE -NetRevenue END),2) FROM sales;
Spark  s.agg(F.round(F.sum(F.when(F.year("OrderDate")==2025, F.col("NetRevenue"))
                          .otherwise(-F.col("NetRevenue"))),2)).show()
```

Section D — Window Functions

D1 — Top 3 products per category

Electronics: Laptop C 33,496,566.90 · Laptop A 29,138,055.96 · Laptop D 24,263,280.30 Furniture: Desk B 15,637,802.63 · Desk D 15,128,861.52 · Desk A 13,995,829.91 Appliances: Kitchen B 15,579,102.40 · Kitchen C 13,024,829.48 · Kitchen D 11,452,666.76 Office Supplies: Paper B 394,954.72 · Binder C 293,539.25 · Paper C 220,669.45

```
SELECT Category, ProductName, r FROM (
  SELECT Category, ProductName, ROUND(SUM(NetRevenue),2) r,
         ROW_NUMBER() OVER (PARTITION BY Category ORDER BY SUM(NetRevenue) DESC) rn
  FROM sales GROUP BY 1,2
) WHERE rn <= 3 ORDER BY Category, r DESC;
```

```
w = Window.partitionBy("Category").orderBy(F.desc("r"))
(s.groupBy("Category","ProductName").agg(F.round(F.sum("NetRevenue"),2).alias("r"))
  .withColumn("rn", F.row_number().over(w)).filter("rn <= 3").orderBy("Category", F.desc("r")).show())
```

Excel: pivot with Category and ProductName in Rows > right-click a Category label > Filter > Top 10 > set to 3.

D2 — Top rep per region

East Rahul Banerjee 30,695,509.36 · North Neha Gupta 28,307,894.08 · South Priya Iyer 31,678,014.81 · West Karan Joshi 31,422,677.98

```
SELECT RegionClean, SalesRep, r FROM (
  SELECT RegionClean, SalesRep, ROUND(SUM(NetRevenue),2) r,
         RANK() OVER (PARTITION BY RegionClean ORDER BY SUM(NetRevenue) DESC) rnk
  FROM sales GROUP BY 1,2) WHERE rnk = 1 ORDER BY 1;
```

```
w = Window.partitionBy("RegionClean").orderBy(F.desc("r"))
(s.groupBy("RegionClean","SalesRep").agg(F.round(F.sum("NetRevenue"),2).alias("r"))
  .withColumn("rnk", F.rank().over(w)).filter("rnk = 1").orderBy("RegionClean").show())
```

D3 — Month-over-month growth (first four months)

2024-01 11,937,752.11 (—) · 2024-02 10,076,030.70 (**−15.60%**) · 2024-03 12,908,730.99 (**+28.11%**) · 2024-04 10,682,769.47 (**−17.24%**)

```
WITH m AS (SELECT strftime(OrderDate,'%Y-%m') mo, SUM(NetRevenue) r FROM sales GROUP BY 1)
SELECT mo, ROUND(r,2),
       ROUND(100.0*(r - LAG(r) OVER (ORDER BY mo)) / LAG(r) OVER (ORDER BY mo), 2) AS mom_pct
FROM m ORDER BY mo;
```

```
m = s.groupBy(F.date_format("OrderDate","yyyy-MM").alias("mo")).agg(F.sum("NetRevenue").alias("r"))
w = Window.orderBy("mo")
m.withColumn("mom_pct", F.round(100*(F.col("r")-F.lag("r").over(w))/F.lag("r").over(w),2)).orderBy("mo").show()
```

> The first row is **NULL**, not zero — there is no prior month. Reporting it as 0% would be wrong.

D4 — Running total, North region

2024-01: 3,373,570.24 → 3,373,570.24 · 2024-02: 2,746,951.45 → 6,120,521.69 · 2024-03: 3,148,257.67 → 9,268,779.36 · 2024-04: 1,967,658.04 → 11,236,437.40

```
SELECT mo, ROUND(r,2),
       ROUND(SUM(r) OVER (ORDER BY mo ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2)
FROM (SELECT strftime(OrderDate,'%Y-%m') mo, SUM(NetRevenue) r
      FROM sales WHERE RegionClean='North' GROUP BY 1) ORDER BY mo;
```

```
w = Window.orderBy("mo").rowsBetween(Window.unboundedPreceding, Window.currentRow)
(s.filter(F.col("RegionClean")=="North")
  .groupBy(F.date_format("OrderDate","yyyy-MM").alias("mo")).agg(F.sum("NetRevenue").alias("r"))
  .withColumn("cum", F.round(F.sum("r").over(w),2)).orderBy("mo").show())
```

D5 — Category share of total

Electronics **51.37%** · Furniture **26.92%** · Appliances **21.10%** · Office Supplies **0.61%**

```
SELECT Category, ROUND(100.0*SUM(NetRevenue)/SUM(SUM(NetRevenue)) OVER (),2)
FROM sales GROUP BY 1 ORDER BY 2 DESC;
```

```
(s.groupBy("Category").agg(F.sum("NetRevenue").alias("r"))
  .withColumn("pct", F.round(100*F.col("r")/F.sum("r").over(Window.partitionBy()),2))
  .orderBy(F.desc("pct")).show())
```

Excel: the pivot's Show Values As > % of Column Total does this in two clicks. > Note `SUM(SUM(x)) OVER ()` — a window over an aggregate. It looks wrong and is correct: the inner `SUM` aggregates the group, the outer window sums across all groups.

D6 — Products above their category average · 21

```
WITH p AS (SELECT Category, ProductName, SUM(NetRevenue) r FROM sales GROUP BY 1,2),
     a AS (SELECT *, AVG(r) OVER (PARTITION BY Category) ca FROM p)
SELECT COUNT(*) FROM a WHERE r > ca;
```

```
p = s.groupBy("Category","ProductName").agg(F.sum("NetRevenue").alias("r"))
p.withColumn("ca", F.avg("r").over(Window.partitionBy("Category"))).filter("r > ca").count()
```

> The CTE layer is **mandatory** — you cannot filter on a window function in `WHERE` or `HAVING`, because windows are computed after both.

D7 — Customers with more than 10 orders · 64

```
SQL    SELECT COUNT(*) FROM (SELECT CustomerID FROM sales GROUP BY 1 HAVING COUNT(*) > 10);
Spark  s.groupBy("CustomerID").count().filter("count > 10").count()
Excel  Pivot CustomerID in Rows, Count in Values, then Value Filter > Greater Than > 10
```

D8 — Reps with zero returns · 0 (none)

```
SQL    SELECT COUNT(*) FROM (SELECT SalesRep FROM sales GROUP BY 1
         HAVING SUM(CASE WHEN Returned='Yes' THEN 1 ELSE 0 END) = 0);
Spark  s.groupBy("SalesRep").agg(F.sum((F.col("Returned")=="Yes").cast("int")).alias("n")).filter("n = 0").count()
```

A zero answer is still an answer — with ~500 orders per rep and a 7% return rate, expecting one is unreasonable.

Section E — Explain

E1. A plain `SUMIFS(...,"South")` misses the rows where Region is `" South"` with a **leading space** — the criteria is an exact string match on the trimmed value. It *does* catch `SOUTH`, because `SUMIFS` and `COUNTIF` are **case-insensitive**. So the defect that bites is whitespace, not casing. Fix with `TRIM` inside `SUMPRODUCT`, or clean the column first.

E2. The 40 duplicate rows are complete copies including revenue. Aggregating before dedup inflates every `SUM` and skews every `AVG`. Deduplication is a *correction*, not a filter — doing it after aggregation is impossible, because the inflation is already baked into the totals.

E3. Excel's `COUNTIF` is **case-insensitive**, so it treats `East` and `EAST` as the same value and reports 8 distinct variants (4 clean + 4 with leading space). SQL's `DISTINCT` is case-**sensitive** and reports all 12. Neither is wrong; they answer different questions. It's a good illustration of why you verify a cleaning step by looking at the values, not by trusting a count.

E4. `AVERAGEIFS` ignores blank cells entirely — they contribute to neither numerator nor denominator. SQL's `AVG()` ignores NULLs the same way. **They agree.** Both differ from `SUM(col)/COUNT(*)`, which would divide by the full row count and understate the average. Being able to state the denominator is the point.

E5. `ROW_NUMBER` guarantees exactly three rows per category. `RANK` would return four or more if there were a tie at third place, breaking the "top 3" contract. Use `RANK` when you want ties included by design; `ROW_NUMBER` when you need a fixed count.

E6. PySpark, on a scheduled job. Excel cannot hold 50M rows at all (1,048,576-row limit). SQL could, if the data is already in a warehouse and properly indexed — that's a legitimate answer. Spark wins when the data sits in a lake, the transformation is complex, or it must scale horizontally. The real answer names the tradeoff rather than picking a favourite.

Section F — Pivot Tables (Excel)

F1. Select `DataClean` > `Ctrl+T` > `Alt+N+V`. RegionClean → Rows, Category → Columns, NetRevenue → Values. Value Field Settings > Number Format > Currency, 0 decimals. *(Add NetRevenue as a helper column first: `=M2*N2*(1-02)`.)*

F2. Right-click any value > **Show Values As > % of Row Total**.

F3. Add OrderDate to Rows > right-click any date > **Group** > select **Years** and **Quarters**. Excel builds the hierarchy automatically. No helper column needed — this is the point of the question.

F4. Drag both Units and NetRevenue into Values. They appear side by side, with a `Σ Values` field you can move between Rows and Columns to change the layout.

F5. PivotTable Analyze > Fields, Items & Sets > **Calculated Field**. Name `MarginPct`, formula `=(NetRevenue - TotalCost)/NetRevenue`. **What it actually computes:** `(SUM(NetRevenue) – SUM(TotalCost)) / SUM(NetRevenue)` — a **revenue-weighted** margin over the group, not the average of the per-order margins. Those differ whenever order sizes vary, which they do here.

F6. Right-click a SalesRep row label > **Filter > Top 10** > change 10 to 5, by Sum of NetRevenue.

F7. ShipDays → Rows > right-click a number > **Group** > Starting at 1, Ending at 12, By 3.

F8. PivotTable Analyze > **Insert Slicer** (Region, Channel) and **Insert Timeline** (OrderDate). Then right-click each slicer > **Report Connections** > tick both pivots. That's the step people miss — without it a slicer drives only its own pivot.

F9. `=GETPIVOTDATA("NetRevenue", $A$3, "RegionClean", "South")` — typing `=` and clicking the cell generates it automatically. Robust to the pivot's layout changing, unlike a plain cell reference.

F10. Excel generates a **new sheet containing the underlying rows** for that cell. Useful for auditing an unexpected number — it's the fastest way to answer "why is this figure so high?"

F11. Changing a UnitPrice then refreshing (`Alt+F5`) updates the pivot. Adding a **row** also works **because the source is an Excel Table** and expands automatically. If the source were a fixed range (`A1:S6001`), the new row would be silently excluded — that's the trap this question exists to teach.

F12. - **Calculated Field** — operates on whole fields, appears in Values. Example: `MarginPct = (NetRevenue – TotalCost)/NetRevenue`. - **Calculated Item** — operates on individual items within one field, appears as a new row/column. Example: inside the Channel field, create `Direct = Online + Retail` as a new item alongside Partner.

Section G — Charts & Dashboard (Excel)

G1. Pivot with OrderDate grouped by Month in Rows, NetRevenue in Values > Insert > Line Chart. Remove gridlines and the legend.

G2. Pivot Category in Rows, NetRevenue in Values, sort descending > Insert > Clustered Column.

G3. Pivot with Category in Rows, NetRevenue and return-rate in Values > Insert > **Combo** chart > set return rate to Line and tick **Secondary Axis**. Needed because revenue is in tens of millions and return rate is a fraction — on one axis the line would be flat against the baseline.

G4. Insert > Scatter with DiscountPct on X and Units on Y. **There is no visible relationship** — consistent with C7, where average units were 3.482 vs 3.493. The chart should confirm your numeric finding, not contradict it.

G5. Select the revenue column > Home > Conditional Formatting > **Top/Bottom Rules > Top 10%**.

G6. Build a small month-by-region grid, then Insert > **Sparklines > Line**, with a location range beside it.

G7. Three sheets: `DataClean` (source), `Pivots` (working), `Dashboard` (output). KPI cards as merged cells driven by `GETPIVOTDATA`. Move charts to the Dashboard sheet (right-click > Move Chart > Object in). Slicers with Report Connections to every pivot. Right-click working sheet tabs > Hide. View > untick Gridlines.

G8. Use a **bar or column chart** sorted descending, or a 100% stacked bar for share. A pie chart is defensible for 4 categories but breaks down at 12 — humans compare angles poorly, small slices become unreadable, and there's no useful ordering. With 12 categories, a sorted bar chart is strictly better.